



UNIVERSITY OF JOHANNESBURG

Blockchain Assignment 7

COLLEGE OF BUSINESS AND ECONOMICS

MASTERS IN FINANCIAL ENGINEERING

Consensus and Forks Lab

Author:

Siphosethu Limane
Keitumetse Motlhabane
Masixole Ntshangase
Joelle Behoor

Student Number:

226214675
220068138
226272432
226259509

INTRODUCTION

A payment system needs a clear and reliable record of who has paid whom and how much was transferred. This record helps people check that a payment has been made and prevents the same amounts from being spent more than once. In a centrally managed system, such as a bank's payment system, the institution maintains the official record and controls how it is updated.

Now a blockchain shares this responsibility of keeping and checking payment records across computers running software called nodes. Each node keeps its own view of the transaction history and checks new information against the network's rules. Nodes exchange transactions and blocks so that they can work towards the same record. However, the information does not reach all nodes at exactly the same time. Things like network delays can cause one node to learn about a new block before another node does.

As a result, nodes may temporarily follow different versions of the blockchain. For example, two miners may create different valid blocks that both follow the same earlier block. Some nodes may receive one block first, while others receive the competing block. This creates a fork, where the chain splits into two possible branches. Neither block is necessarily incorrect, but the nodes need a rule to decide which branch to follow.

Assignment 7 explores this situation by simulating three nodes, named A, B and C. Each node holds its own chain, allowing the simulation to show how their records can differ. Competing valid blocks are created, and a fork choice rule is applied to resolve the disagreement. With the longest chain rule, nodes select the valid branch containing the most blocks. The assignment also considers what happens when branches have equal lengths or when a competing branch arrives late.

These situations matter because a payment on one branch may no longer be in a node's record when it changes to another branch. We call this change a chain reorganisation. So then seeing a payment in a block does not mean it is permanent. Confirmations count the block containing the payment and each new block added after it on the same chain. Waiting for more confirmations can make a reversal less likely under normal network conditions, but it cannot guarantee that the payment will never be reversed.

The assignment connects these ideas to a cross border retail payment of R10 000 equivalent. A

recipient or payment provider must decide how long to wait before accepting the payment or releasing local currency. Accepting it quickly reduces waiting time but may increase exposure to a reversal. Waiting for more confirmations provides additional protection but delays access to the funds. Through the simulation and supporting research, the assignment examines how this balance between speed and payment security can guide a confirmation policy.

PART A

A. Simulating three node

The first part of the code simulates three blockchain nodes, called A, B, and C. Each node has its own copy of the blockchain. The code first creates a genesis block, which is the first block in the chain. The genesis block is mined using Proof-of-Work with a difficulty of 3, meaning that the block hash must start with three zeros. The genesis block produced a hash starting with 0009693e594e4643, with a nonce of 449.

After this, three nodes were created. At the beginning, all three nodes had the same genesis block. The results were:

```
A: height=0 tip=0009693e59... valid=True work=3
B: height=0 tip=0009693e59... valid=True work=3
C: height=0 tip=0009693e59... valid=True work=3
Shared tip? True
```

This shows that A, B, and C initially agreed on the same blockchain. All three nodes had a valid chain, the same tip, and the same amount of work. The important point is that each node stores its own copy of the chain, so the nodes can later have different versions of the blockchain.

B. Creating a fork

A fork was created by having nodes A and B mine different blocks from the same genesis block. Node A mined a block containing a R10,000 payment to Alice, while node B mined a different block containing a R10,000 payment to Bob. Since both blocks used the same genesis block as their parent, they created two competing chains.

The results were:

```
A and B tips differ? True
Both extensions valid? True | True
Same parent (genesis)? True
```

This confirms that both blocks were valid and that they came from the same parent. The chains can therefore be represented as

$$G \rightarrow 1_A$$

and

$$G \rightarrow 1_B.$$

At this point, A and B both had a chain length of 2 and cumulative work of 6, while C was still at the genesis block with a length of 1 and work of 3. This created a tie between A and B because both chains had the same amount of work. The code resolves forks using cumulative work. When two chains have the same amount of work, a deterministic tie-breaking rule based on the block hash is used. In this case, the chain containing the payment to Alice was selected. The result was also independent of the order in which the chains were provided to the function. The payment to Bob was therefore not included in the winning chain.

The code also tested what happens when a longer chain is invalid. An invalid chain was created and made longer than the valid chain. However, it was rejected because its hash did not match the block data. The result was:

```
Invalid longer chain rejected; winner height: 1
Winner valid? True
```

This shows that a chain cannot win simply because it is longer. It must first pass the blockchain validation checks. The code also compares the longest-chain rule with the heaviest-work rule. For example, a chain with five easy blocks has a length of 5 and work of 5, while a chain with three harder blocks has a length of 3 and work of 9. The longest-chain rule would select the first chain, while the heaviest-work rule would select the second. In this assignment, all blocks use the same difficulty, so the two methods give the same result.

C. Edge cases

The first edge case considered is what happens when one node extends its chain after a fork. Node A mined another block after the fork, giving A a chain with more cumulative work than B's chain. Before synchronisation, the nodes had different views:

```
A: height=2 tip=000980015e... valid=True work=9
B: height=1 tip=00091fec5d... valid=True work=6
C: height=0 tip=0009693e59... valid=True work=3
```

A therefore had the strongest chain because it had the greatest cumulative work. After synchronisa-

tion, all three nodes adopted A's chain:

```
A: height=2 ... work=9
B: height=2 ... work=9
C: height=2 ... work=9
```

The shared tip was the block from A's chain, and the canonical payment was `pay-alice`. This shows how nodes can eventually agree on one chain after temporarily having different versions. Another edge case is an orphaned block. Node B's block containing `pay-bob` was valid when it was created, but it was not part of the winning chain after A's chain became heavier. The code showed that the block was still a valid extension of the genesis block, but it was no longer part of the canonical chain. Therefore, the payment to Bob had zero confirmations and would need to be included in a new block before becoming part of the accepted chain. The code also demonstrates delayed block arrival. Node C initially only knew about the chain from B, so it adopted B's chain and showed `pay-bob` as its local payment. When C later received A's heavier chain, it changed from B's chain to A's chain. The result showed `reorg occurred? True`. This is an example of a chain reorganisation, where a node changes its local blockchain after receiving a stronger competing chain.

Finally, the code considers confirmation depth for a R10,000 payment. Confirmation depth refers to how many blocks are supporting a transaction after it has been included in the blockchain. If a transaction is included at height h and the current chain height is H , the number of confirmations is calculated as

$$k = H - h + 1.$$

In the simulation, `pay-alice` was included at height 1. When the chain reached height 2, the payment had 2 confirmations. More blocks were then added until the chain reached height 5, giving the payment 5 confirmations. The result was:

```
pay-alice confirmations now: 5
coffee-shop (k>=1): True
class policy (k>=3): True
OTC desk (k>=6): False
```

For the R10,000 payment, a classroom policy of $k = 3$ confirmations was chosen. This means the payment would be treated as settled after three confirmations in this simulation. The reason for waiting for more confirmations is that each additional block makes it harder for a previous transaction to be removed through a chain reorganisation. However, the difficulty used in this assignment is only

a small classroom example and should not be treated as providing real-world blockchain security. Overall, the simulation shows how three blockchain nodes can start with the same chain, temporarily disagree when a fork occurs, and later reach agreement when one chain becomes stronger. It also shows how ties, delayed blocks, orphaned transactions, and confirmation depth can affect the state of a blockchain.

PART B

Block Relay and Network Synchronisation

As previously noted bitcoin operates as a peer-to-peer network in which full nodes independently validate and maintain a copy of the blockchain. When a miner successfully produces a new block, the block is propagated through the network and validated by receiving nodes before being relayed further. The blockchain therefore represents an ordered sequence of blocks, with each block containing a reference to its predecessor through the previous-block hash (Antonopoulos and Harding, 2023).

Block relay time is really important because the network is distributed rather than centrally controlled. This means that a newly mined block cannot be observed by every node immediately. During the period in which a block is propagating, different miners may temporarily have different views of the most recent blockchain tip. If two valid blocks extending the same parent are produced before the network has fully synchronised, competing branches can temporarily exist. This is why mining is important because the Bitcoin consensus mechanism subsequently causes nodes to converge on the branch with the greatest accumulated proof-of-work. This provides a direct link between block propagation and the fork simulation presented in Part A.

Empirical research demonstrates that block propagation is generally measured on a much shorter timescale than the Bitcoin block-generation interval, although the precise delay depends on network conditions and the relay protocol. For example, empirical measurements of Bitcoin-related block relay have reported substantial improvements from the use of compact-block relay, while other experimental work demonstrates that propagation delays can vary according to network connectivity and protocol conditions (Nagayama et al., 2020; Imtiaz et al., 2022). Consequently, block relay time should be interpreted as a distribution rather than as a fixed deterministic value.

Confirmation Depth

A transaction initially enters the network as an unconfirmed transaction. Once a miner includes the transaction in a valid block, it has received one confirmation. Each subsequent block added to the same chain increases the transaction's confirmation depth by one. Thus, if a transaction is included in block B_t , then the number of confirmations can be expressed as

$$C = 1 + n, \tag{3.1}$$

where C is the confirmation count and n is the number of blocks subsequently built on top of the block containing the transaction.

Confirmation depth is important because a transaction contained in the most recent block may still be affected by a temporary chain reorganisation. As additional blocks are added, the transaction becomes increasingly buried beneath subsequent proof-of-work.

It is important to distinguish *block relay time* from *confirmation time*. Relay time describes how quickly a particular block is distributed through the network, whereas confirmation depth describes how many blocks have been built on top of the block containing the transaction. A block may propagate through the network within seconds, but obtaining additional confirmations requires additional blocks to be mined. Therefore, reducing propagation delay does not eliminate the need for a confirmation policy.

Remittance Application

Let's look at something practical when thinking of remittance. If we have a cross-border remittance in which a sender transfers the equivalent of R10,000 in Bitcoin to a recipient. The recipient's payment policy must balance settlement speed against the risk of accepting a transaction that could subsequently be removed from the chain through a reorganisation.

At zero confirmations, the transaction has been broadcast but has not yet been included in a block. Accepting the payment at this stage provides the fastest possible settlement, but exposes the recipient to much more uncertainty. Once the transaction receives one confirmation, it has been included in a valid block, but that block remains the most recent block and can still potentially be replaced. Further confirmations increase the amount of subsequent proof-of-work that would need to be overcome for the transaction to be removed from the selected chain.

A confirmation policy can therefore be represented as a trade-off:

$$\text{Settlement speed} \longleftrightarrow \text{Confidence in transaction finality}. \tag{3.2}$$

Therefore we can say a remittance provider could define different operational thresholds according to the value and risk of the payment:

The thresholds in Table 3.1 are illustrative rather than actual global requirements. The appropriate

Table 3.1: Illustrative Confirmation Policy for an R10,000 Remittance

Confirmations	Settlement Status	Interpretation
0	Unconfirmed	Payment has been broadcast but is not yet included in a block. Suitable only where the recipient accepts significant settlement risk.
1	Initial confirmation	Transaction is included in a block, but the block is still the chain tip and may be replaced by a competing block.
2–3	Increasing confidence	Additional blocks provide greater protection against a short chain reorganisation, at the cost of additional waiting time.
6	Conservative confirmation threshold	The transaction is buried beneath multiple subsequent blocks, providing substantially greater resistance to an ordinary short reorganisation.

policy depends on the remittance provider's tolerance for settlement risk, the transaction value, the required speed of payment, and the prevailing network conditions.

Block Relay Time versus Confirmation Policy

Figure 3.1 shows us the relationship between propagation and confirmation policy. When a miner discovers a block, the block first propagates through the network. During this period, nodes may temporarily observe different chain tips. Once the transaction's block has propagated and been accepted by the network, the next blocks increase its confirmation depth.

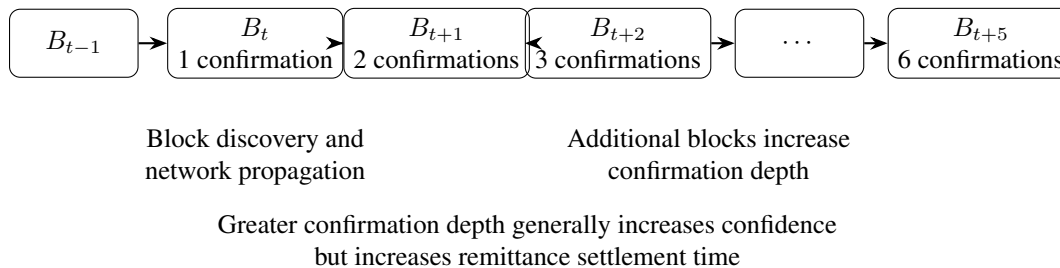


Figure 3.1: Relationship between block propagation, confirmation depth and remittance settlement policy.

The figure highlights an important distinction. Block propagation is a network communication process, whereas confirmation depth is a consensus and risk-management measure. Even if a block is rapidly relayed across the network, the recipient may still require additional confirmations before releasing the R10,000 remittance. Conversely, imposing a very high confirmation threshold improves settlement confidence but delays the availability of funds.

This trade-off is particularly relevant to cross-border payments. A remittance provider may value rapid settlement because recipients may require immediate access to funds, but accepting a payment before sufficient confirmation exposes the provider to the possibility that the transaction is not retained in the eventual canonical chain. The confirmation policy therefore converts the technical properties of Bitcoin's consensus mechanism into an operational risk-control decision.

Link to the Fork Simulation

The analysis in Part B extends the fork scenario implemented in Part A. In the simulation, two valid competing blocks can temporarily be observed by different nodes. This is analogous to the temporary divergence that can occur when competing blocks are produced and have not yet been fully propagated through the network. The longest-chain or greatest-work rule subsequently determines which branch becomes authoritative.

For the R10,000 remittance, confirmation depth provides an operational response to this temporary uncertainty. A payment with one confirmation remains close to the chain tip and is therefore more exposed to a short reorganisation than a payment buried beneath several subsequent blocks. Waiting

for additional confirmations does not make a transaction mathematically irreversible; rather, it makes reversal through an ordinary competing chain progressively more difficult.

Thus, the consensus process can be viewed as a sequence:

Transaction broadcast → Block inclusion → Block propagation → Chain convergence →
Additional confirmations → Remittance settlement.

The central implication is that block relay speed and confirmation depth address related but different aspects of settlement. Efficient relay reduces the period during which nodes have inconsistent views of the latest block, while additional confirmations provide increasing confidence that the transaction will remain part of the accepted blockchain.

PART C

Problem Framing

Nakamoto (2008) addresses the double-spending problem in digital payments: how two parties can transfer electronic value without relying on a trusted financial intermediary to maintain a single authoritative transaction history. The proposed solution combines digital signatures with a peer-to-peer network, a hash-linked transaction history, and proof-of-work. The network accepts the chain containing the greatest accumulated proof-of-work as the authoritative history, provided that honest participants control the majority of computational power (Nakamoto, 2008).

From a regulated remittance perspective, the relevant settlement hazard is that a transaction initially regarded as settled may later be displaced by an alternative chain. Nakamoto addresses this risk probabilistically: as additional blocks are added after a transaction, the probability that an attacker can catch up with the honest chain decreases (Nakamoto, 2008). The relevant stakeholders therefore extend beyond the sender and recipient to include the remittance provider, miners, financial institutions and regulators. For a retail payment such as an R10 000 remittance, the key question is whether the paper's assumptions and evidence are sufficient to establish an acceptable level of settlement certainty for a regulated financial environment.

Mechanism Audit

Nakamoto's security argument rests on several assumptions that are stronger than the protocol description itself. The central assumption is that honest participants control more computational power than an attacking coalition. Under this condition, an attacker attempting to replace a confirmed transaction must generate an alternative chain faster than the honest network. The paper models this competition as a random walk, with the honest and attacker block-finding probabilities represented by p and q , respectively (Nakamoto, 2008).

The network model assumes sufficiently effective propagation for participants to converge on the same transaction history, although communication is explicitly described as best effort. Temporary disagreement is acknowledged: if two blocks are produced at approximately the same time, different nodes may initially follow different branches. The longest-chain rule resolves this fork when a

subsequent block causes one branch to become longer (Nakamoto, 2008). However, the paper does not provide an empirical model of propagation delay, geographic network structure, or prolonged network partitions.

Participant behaviour is treated primarily through economic incentives rather than a formal model of strategic behaviour. Nakamoto (2008) argues that an attacker may find honest mining more profitable than attempting to undermine the system, but this remains an incentive-based argument rather than a result demonstrated across different market conditions or attacker objectives (Nakamoto, 2008).

The strongest analytical component is the calculation of the attacker's probability of catching up with the honest chain. Given p , q , and confirmation depth z , the paper derives the probability that an attacker can eventually overtake the honest chain and shows that this probability declines as z increases when $p > q$. The numerical results evaluate this relationship for different attacker shares and confirmation depths (Nakamoto, 2008). These results are analytical calculations rather than empirical measurements of a live payment network. Thus, the mechanism is theoretically coherent, but important assumptions concerning propagation, participant incentives, and attacker behaviour remain insufficiently tested.

Evidence and Limits

The paper's conclusions would be strengthened by empirical evidence showing that its assumptions remain valid under conditions representative of retail payments. Firstly, data on the distribution of computational power across miners and mining pools would be required to assess whether the assumed honest majority is stable over time. This is particularly important because the calculated probability of successful chain reversal depends directly on the attacker's share of computational power, q (Nakamoto, 2008). Secondly, measurements of transaction and block propagation times, together with stale-block and chain-reorganisation frequencies, would provide evidence on whether network conditions materially affect the longest-chain mechanism.

For an R10 000 remittance, additional evidence should relate the technical risk to the economic value of the payment. This would include observed confirmation times, transaction fees, the frequency and depth of chain reorganisations, and the historical incidence of transactions being reversed after increasing numbers of confirmations. Such evidence would strengthen Nakamoto's conclusions if observed reversal risks remained sufficiently low under realistic network conditions and if computational power remained sufficiently distributed. Conversely, evidence of substantial mining concentration, frequent reorganisations, or propagation conditions that increase the probability of competing chains would weaken the applicability of the theoretical results.

The main limitation is therefore not that the paper fails to provide a mathematical security argument, but that the argument is conditional on assumptions that are not established through empirical payment-system data. For a regulated remittance application, technical confirmation depth would therefore need to be assessed alongside the operational and financial consequences of an unsuccessful settlement.

Comparison

Nakamoto's proof-of-work mechanism provides probabilistic finality. A transaction becomes increasingly difficult to reverse as additional blocks are added to the chain, but the possibility of an alternative chain is not eliminated completely (Nakamoto, 2008). This differs from Byzantine Fault Tolerant (BFT) designs such as Practical Byzantine Fault Tolerance (PBFT), where a known set of replicas reaches agreement through a quorum-based protocol and a committed operation can be treated as final under the protocol's fault assumptions (Castro and Liskov, 1999).

The difference is therefore not simply one of speed, but of how settlement certainty is established. Nakamoto replaces reliance on a trusted intermediary with computational competition and probabilistic confirmation. A BFT design instead relies on an identified set of participating replicas and an assumption that the number of Byzantine participants remains within the protocol's fault tolerance. For a regulated remittance rail, this changes the settlement model from waiting for sufficiently high confirmation confidence to obtaining protocol-level agreement from the validator set. However, this comparison does not establish that either approach is universally preferable; the appropriate design depends on the governance, fault, and operational requirements of the payment system.

Open Questions

Two testable questions follow from the limitations identified above. Firstly, how does the relationship between block propagation time and block-generation time affect the frequency of competing chains and the confirmation depth required to achieve a specified level of settlement confidence? This could be tested using historical blockchain network data containing block timestamps, propagation measurements, and chain-reorganisation events (Decker and Wattenhofer, 2013).

Secondly, how does increasing concentration of computational power affect the empirical probability of successful chain reversal for transactions of different values, including R10 000 retail remittances? This question could be investigated by combining historical mining-pool shares with simulated attack probabilities and observed network conditions. It would extend Nakamoto's theoretical treatment of the attacker's computational share q by examining how changes in mining concentration affect

practical settlement risk (Nakamoto, 2008; Eyal and Sirer, 2014).

Verdict

Revise

Nakamoto's paper provides a coherent theoretical mechanism for reducing double-spending without relying on a central financial intermediary. Its proof-of-work mechanism, longest-chain rule, and analysis of attacker catch-up probability establish a clear relationship between computational power, confirmation depth, and the probability of transaction reversal. However, the evidence presented is not sufficient on its own to establish the suitability of the mechanism as a regulated remittance rail.

The principal reason for revision is that several conditions underlying the security analysis are treated as assumptions rather than empirically established properties of a regulated payment environment. In particular, the analysis depends on the distribution of computational power, effective network propagation, and assumptions about participant behaviour. The paper also does not evaluate settlement risk specifically for retail remittances or address the operational and governance requirements that would accompany regulated financial infrastructure.

This verdict does not claim that proof-of-work cannot provide secure settlement, nor that Nakamoto's theoretical analysis is invalid. It instead means that additional empirical evidence and an explicit treatment of operational and regulatory requirements would be required before the mechanism could be assessed as an appropriate design for regulated remittance rails.

DISCLOSURES

5.1 Contribution statement

Joelle assisted with Part B of the assignment, while Masixole assisted with Part C. Siphosethu and Keitumetse worked on Part A and were also responsible for checking, debugging, and ensuring that the code runs correctly. All group members contributed to the completion and review of the final report.

5.2 AI Disclosure

AI was used to assist with the development and debugging of the code. It was mainly used to identify errors, explain parts of the code, and suggest corrections.

5.3 Github link

Be careful with the dash after py. If it fails when clicking please try adding dash after py.

<https://github.com/Motlhabane-Keitu/Blockchain-Assignment-7-Consensus-and-Forks-Lab-consensus.py-.git>

BIBLIOGRAPHY

- Andreas M. Antonopoulos and David A. Harding. *Mastering Bitcoin: Programming the Open Blockchain*. O'Reilly Media, 3 edition, 2023.
- Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation*, pages 173–186, 1999.
- Christian Decker and Roger Wattenhofer. Information propagation in the bitcoin network. In *IEEE P2P 2013 Proceedings*, pages 1–10. IEEE, 2013.
- Ittay Eyal and Emin Gün Sirer. Majority is not enough: Bitcoin mining is vulnerable. In *International Conference on Financial Cryptography and Data Security*, pages 436–454. Springer, 2014.
- Muhammad Anas Imtiaz, David Starobinski, and Ari Trachtenberg. Empirical comparison of block relay protocols. *IEEE Transactions on Network and Service Management*, 19(4):3960–3974, 2022. doi: 10.1109/TNSM.2022.3195976.
- Ryunosuke Nagayama, Kazuyuki Shudo, and Ryohei Banno. Identifying impacts of protocol and internet development on the bitcoin network. In *2020 IEEE Symposium on Computers and Communications (ISCC)*, 2020. doi: 10.1109/ISCC50000.2020.9219639.
- Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2008. URL <https://bitcoin.org/bitcoin.pdf>.